En la empresa ficticia “Excusas S.A.”, un grupo de empleados se dedica exclusivamente a gestionar las excusas que los trabajadores presentan cuando llegan tarde. De cada empleado sabemos su nombre, su email y su número de legajo. De cada excusa sabemos cual es el empleado que llega tarde, el motivo por el cual llego tarde (que solo puede ser quedarse dormido, perdí un transporte, pérdida de suministro, cuidado de familiar a cargo, irrelevante e increíble o inverosímil). Una excusa puede estar aceptada o no. Existen distintos tipos de excusas para llegar tarde.

<<Para automatizar el sistema, la empresa decidió que cada tipo de excusa será evaluada por un sector específico. Si la puede procesar, la procesa y si no, continúa su tratamiento en otro sector

Considere que los responsables de estos sectores implementan esta interfaz:

public void revisarExcusa(Excusa unaExcusa);

Cada responsable es también un empleado por lo que comparten ciertas características. Las excusas atraviesan distintos niveles organizacionales hasta encontrar quien decida intervenir o finalizar el proceso Cada responsable debe decidir si procesa la excusa o la deriva al siguiente en sector. Estos son: >> → Chain of Responsibility

Me pareció Chain of Responsibility porque la excusa va pasando de un sector a otro según su capacidad de procesamiento: si el responsable puede, la procesa y, si no, la deriva al siguiente. Todos los responsables comparten la misma interfaz (revisarExcusa) y cada uno conoce solo a su siguiente, por lo que quien presenta la excusa solo habla con el primero de la cadena. Si nadie puede procesarla, se devuelve la respuesta de rechazo del enunciado.

1. Recepcionista: Acepta excusas triviales como me quedé dormido o perdí el colectivo. Cuando las acepta, envía un email al empleado de asunto “motivo demora” diciendo que “la licencia fue aceptada”.

2. Supervisora de Área: Acepta excusas moderadas como se cortó la luz en todo el barrio o tuve que cuidar a un familiar, donde envía un email a EDESUR@mailfake.com.ar consultando si esto es verdad en el primer caso. Si es el segundo, le envía un email al empleado preguntando si todo está bien.

3. Gerente de Recursos Humanos: Acepta excusas complejas como fui abducido por aliens o una paloma robó mi bicicleta.

4. CEO: Solo escucha excusas extremadamente inverosímiles. Siempre responde con: “Aprobado por creatividad” al email del empleado. Cuando esto sucede, se empieza un prontuario con los datos del empleado, la excusa y el nro de legajo. Por otro lado, debemos considerar que:

<<1) Con este fin, existe un Administrador de Prontuarios, que se encarga de persistir los prontuarios. La gestión de registros especiales debe centralizarse para evitar inconsistencias. >> → Singleton

Me pareció Singleton porque el enunciado insiste en que la gestión de prontuarios se centralice para evitar inconsistencias, y eso se logra asegurando que exista una única instancia del Administrador de Prontuarios, con un punto de acceso global al que recurre el CEO cuando registra un prontuario.

2)<< Cuando un CEO releva un nuevo prontuario se mantiene informado al equipo de dirección sobre los cambios relevantes en los registros . >> → Observer

Me pareció Observer porque cuando el CEO registra un prontuario hay que avisar al equipo de dirección sin que el Administrador de Prontuarios dependa de quién recibe el aviso. El Administrador es el sujeto: mantiene una lista de observadores y los notifica cada vez que se registra un prontuario; el equipo de dirección es un observador. Así se pueden sumar otros interesados sin modificar al Administrador. El Administrador cumple a la vez el rol de Singleton y el de sujeto, pero cada patrón resuelve un problema distinto.

3) Considerar que el orden de los responsables en la organización del procesamiento de las excusas puede y debe ser absolutamente configurable (es decir, que encargadxs la conforman y en qué orden) → Builder

Me pareció Builder porque la cadena tiene que poder armarse con los responsables que se quieran y en el orden que se quiera. Un constructor (builder) va encadenando los responsables en el orden indicado y entrega la cadena armada, sin que los responsables conozcan su posición. Para sumar un responsable nuevo, como “el coach motivacional”, alcanza con crearlo y agregarlo a la cadena: no se modifica nada de lo existente (principio abierto/cerrado).

<<Además, el comportamiento de un responsable puede verse modificado temporalmente según su nivel de desempeño laboral alterando la forma particular de resolver el procesar las excusas. Puede ser normal, vagx o productivx. Si es normal, hace lo que se espera. Si es vago no lo evalúa y lo pasa a otro sector. En cambio, si está en modo productivo lo evalúa si puede hacerlo o lo pasa a otro sector si no puede, no sin antes enviarle un email al CTO de la empresa.

A priori, cuando un responsable procesò menos de 5 excusas está Productivx. Si procesó más de 5 excusas pero menos de 10 está Normal. Si proceso 11 excusas o más está Vagx. Un responsable volverá a estar Productivo cuando tomé un café, Normal cuando se toma un break de entre 10 y 15 minutos (Si toma menos o más, no surte efecto) Cuando esté vago y vuelva al trabajo al otro día tiene 50% de posibilidades de volverse Productivo o Normal. El sistema debe ser capaz de agregar distintos modos para los Encargados en tiempo de ejecución que puedan surgir en un futuro. >> → Strategy

Me pareció Strategy porque cada modo de desempeño (normal, vago y productivo) es una forma distinta e intercambiable de resolver una excusa. El responsable (contexto) delega en el modo que tiene asignado y ese modo se puede reemplazar en tiempo de ejecución, por lo que se pueden agregar modos nuevos sin modificar al responsable ni a los modos existentes. Las reglas de cuándo se cambia de modo (cantidad de excusas, café, break y azar del día siguiente) se separan de los modos: así ningún modo conoce a los otros, cosa que no pasaría si cada uno decidiera a cuál pasar, ya que sumar un modo obligaría a modificar los existentes.

Para enviar un email, asuman que cuentan con la clase EmailSender que implementa esta interfaz:

public void enviarEmail(String unEmailDestino, String unEmailOrigen, String unAsunto, String unCuerpo)

Si nadie puede procesarla, se devuelve una respuesta: “Excusa rechazada: necesitamos pruebas contundentes”.